

ROTEIRO DE SIMULAÇÃO
MODULAÇÕES DIGITAIS UNIPOLARES NRZ-L E NRZ-I EM VHDL NO VIVADO

PROJETO DE COMUNICAÇÃO DE DADOS E SISTEMAS RECONFIGURÁVEIS

SÃO BERNARDO DO CAMPO

2026

1. INTRODUÇÃO

Este roteiro apresenta, de forma detalhada, os procedimentos necessários para criar, configurar, executar e interpretar dois projetos independentes no Vivado, ambos implementados em VHDL para simulação de modulações digitais unipolares. O primeiro projeto corresponde à modulação NRZ-L, na qual o nível do sinal de saída acompanha diretamente o valor do bit de entrada. O segundo corresponde à modulação NRZ-I, na qual o bit 1 provoca inversão do nível do sinal e o bit 0 mantém o nível anterior.

O objetivo principal do presente documento é permitir que um usuário sem experiência prévia com VHDL ou com a IDE Vivado consiga abrir os projetos, adicionar os arquivos, executar a simulação comportamental e compreender a relação entre a entrada binária e a forma de onda gerada. O roteiro também interpreta os resultados obtidos nas formas de onda anexadas, identificando os sinais principais e relacionando o comportamento observado com as regras de cada modulação.

2. ORGANIZAÇÃO DOS ARQUIVOS UTILIZADOS

Os projetos estão organizados em duas pastas independentes. A separação é importante porque cada modulação possui uma entidade principal e um testbench próprios. No projeto NRZ-L, a entidade principal é `nrz_l.vhd` e o testbench é `tb_nrz_l.vhd`. No projeto NRZ-I, a entidade principal é `nrz_i.vhd` e o testbench é `tb_nrz_i.vhd`.

PROJETO	ARQUIVO	FUNÇÃO NO PROJETO
Projeto NRZ-L	<code>nrz_l.vhd</code>	Source com a implementação da modulação NRZ-L.
Projeto NRZ-L	<code>tb_nrz_l.vhd</code>	Testbench responsável por gerar clock, reset, start e entrada de bits para a simulação.
Projeto NRZ-I	<code>nrz_i.vhd</code>	Source com a implementação da modulação NRZ-I por inversão de nível.
Projeto NRZ-I	<code>tb_nrz_i.vhd</code>	Testbench responsável por aplicar a sequência de bits e permitir a visualização da forma de onda NRZ-I.

Embora a placa de referência seja a Basys 3, nesta etapa o foco é a simulação comportamental. Assim, não é necessário gerar bitstream nem implementar o circuito fisicamente na placa FPGA. O mais importante é validar, por meio do Vivado Simulator, se a saída modulada corresponde à regra definida para cada técnica de codificação.

3. PARÂMETROS GERAIS DOS PROJETOS

Nos dois projetos, a entrada de dados foi modelada como um vetor de 16 bits, compatível com a quantidade de switches disponíveis na placa Basys 3. O sinal *data_in* possui índices de 15 até 0, e a transmissão foi definida do bit mais significativo para o menos significativo, isto é, de *data_in*(15) até *data_in*(0).

O clock da simulação possui período de 20 ns. A duração de cada bit, também chamada de intervalo de símbolo, foi definida como 100 ns. Portanto, cada bit permanece representado na saída durante 5 ciclos de clock, conforme o cálculo apresentado a seguir:

```
CLK_PERIOD = 20 ns  
BIT_PERIOD = 100 ns  
CYCLES_PER_BIT = BIT_PERIOD / CLK_PERIOD = 100 ns / 20 ns = 5
```

Essa relação é importante porque o circuito principal não trabalha diretamente com unidades de tempo, mas com ciclos de clock. O tempo em nanossegundos aparece no testbench, enquanto o source recebe a quantidade de ciclos por bit por meio de *generic*. Dessa forma, o circuito permanece síncrono e parametrizável.

A sequência de entrada é definida no testbench pela constante *INPUT_BITS*. Nas formas de onda analisadas neste roteiro, a sequência visualizada é 1011001110001111. Caso a sequência seja alterada no testbench, a forma de onda gerada também será alterada, mas o princípio de interpretação permanece o mesmo.

4. CRIAÇÃO DO PROJETO NO VIVADO

O procedimento de criação deve ser repetido duas vezes: uma para o projeto NRZ-L e outra para o projeto NRZ-I. A recomendação é criar pastas separadas, por exemplo, *NRZ_L* e *NRZ_I*, para evitar conflito entre nomes de entidades, testbenches e arquivos de simulação.

Para iniciar, abra o Vivado e selecione a opção *Create Project*. Na janela inicial do assistente, avance para a etapa de definição do nome do projeto. Para o primeiro projeto, utilize um nome como *NRZ_L*. Para o segundo, utilize um nome como *NRZ_I*. Marque a opção *RTL Project*, pois o projeto será descrito por código VHDL.

Na etapa de seleção do dispositivo, escolha a placa Basys 3, caso o arquivo de placa esteja instalado no Vivado. Caso a placa não apareça, selecione diretamente o componente *xc7a35tcpg236-1*, correspondente à família Artix-7 utilizada pela Basys 3. Após concluir o assistente, o Vivado abrirá o ambiente do projeto vazio, pronto para receber os arquivos VHDL.

5. ADIÇÃO DOS ARQUIVOS VHDL

Com o projeto aberto, utilize a opção *Add Sources*. Para o arquivo principal, selecione *Add or Create Design Sources*. No projeto NRZ-L, adicione o arquivo `nrz_l.vhd`. No projeto NRZ-I, adicione o arquivo `nrz_i.vhd`. Esses arquivos representam a lógica sintetizável de cada modulação.

Em seguida, adicione o testbench por meio da opção *Add or Create Simulation Sources*. No projeto NRZ-L, adicione `tb_nrz_l.vhd`. No projeto NRZ-I, adicione `tb_nrz_i.vhd`. O testbench não será implementado na FPGA; ele serve apenas para gerar os estímulos necessários à simulação.

Depois de adicionar os arquivos, confira no painel Sources se o source aparece dentro de Design Sources e se o testbench aparece dentro de Simulation Sources. Essa separação é essencial para que o Vivado reconheça corretamente qual código descreve o circuito e qual código serve somente para simulação.

6. CONFIGURAÇÃO DA SIMULAÇÃO

A simulação utilizada é a Behavioral Simulation, pois o objetivo é verificar o funcionamento lógico do código antes de qualquer etapa de síntese ou implementação. No painel *Flow Navigator*, acesse *Simulation* e selecione *Run Simulation*, depois *Run Behavioral Simulation*.

Antes de executar, é recomendável abrir as configurações de simulação e verificar o tempo de execução. Como a sequência possui 16 bits e cada bit dura 100 ns, a transmissão completa consome 1600 ns somente para os dados. Como ainda existem tempos de reset, start e margem final, recomenda-se simular pelo menos 1700 ns ou 1800 ns. Nas capturas anexadas, a simulação foi observada em torno de 1700 ns.

Na janela de waveform, devem ser adicionados os sinais `clk`, `rst`, `start`, `data_in`, `nrz_out` e `bit_indx`. O sinal `data_in` identifica a entrada de bits; o sinal `nrz_out` identifica a saída modulada; e o sinal `bit_indx` mostra qual posição do vetor está sendo processada em cada intervalo de bit.

SINAL	TIPO	FINALIDADE NA SIMULAÇÃO
clk	Entrada	Clock síncrono do circuito, com período de 20 ns.
rst	Entrada	Reinicia os registradores internos e retorna o índice ao bit 15.
start	Entrada	Pulso de início que autoriza a transmissão da sequência.

SINAL	TIPO	FINALIDADE NA SIMULAÇÃO
data_in[15:0]	Entrada	Vetor de bits a ser modulado, lido do MSB para o LSB.
nrz_out	Saída	Sinal modulado resultante da regra NRZ-L ou NRZ-I.
bit_indx	Saída auxiliar	Indica o índice do bit atualmente associado ao sinal de saída.

7. EXECUÇÃO DA SIMULAÇÃO

Após a abertura da simulação comportamental, o Vivado compila os arquivos e carrega o snapshot do testbench. Em seguida, a simulação pode ser executada pelo botão Run For, informando o tempo desejado, por exemplo, 1700 ns. Outra possibilidade é utilizar o console Tcl com o comando run 1700 ns.

O testbench inicialmente coloca *rst* em nível alto, forçando o circuito a um estado conhecido. Depois, *rst* retorna para zero, e o sinal start é acionado por um ciclo de clock. A partir do momento em que o circuito captura o start em uma borda de subida do clock, a saída modulada começa a representar a sequência de bits.

O sinal *bit_indx* deve aparecer em ordem decrescente: 15, 14, 13, ..., 0. Cada valor de *bit_indx* permanece ativo por 100 ns, que correspondem a 5 ciclos de clock. Dessa forma, fica visualmente simples associar cada trecho da onda de saída ao bit correspondente do vetor *data_in*.

8. PROJETO NRZ-L

8.1 Princípio de funcionamento

Na modulação Unipolar NRZ-L, o bit é representado diretamente pelo nível do pulso durante todo o intervalo de bit. Assim, quando o bit de entrada é 1, a saída permanece em nível alto durante 100 ns. Quando o bit de entrada é 0, a saída permanece em nível baixo durante 100 ns.

A sigla NRZ significa Non-Return-to-Zero. Isso indica que o sinal não retorna obrigatoriamente para zero no meio do intervalo de bit. O nível permanece constante durante todo o período do símbolo, mudando apenas quando o próximo bit exige outro nível.

8.2 Trechos relevantes do source nrz_l.vhd

O trecho a seguir mostra os parâmetros genéricos e as portas principais da entidade. O generico *N_BITS* define o tamanho do vetor de entrada, enquanto *CYCLES_PER_BIT* define por quantos ciclos de clock cada bit será mantido na saída.

```

entity nrz_1 is
  generic (
    N_BITS      : integer := 16;
    CYCLES_PER_BIT : integer := 5
  );
  port (
    clk      : in  std_logic;
    rst      : in  std_logic;
    start    : in  std_logic;
    data_in  : in  std_logic_vector(N_BITS - 1 downto 0);
    nrz_out  : out std_logic;
    bit_indx : out integer range 0 to N_BITS - 1
  );
end nrz_1;

```

A estrutura interna utiliza registradores para armazenar o índice do bit, a contagem de ciclos e o valor atual da saída. O sinal *bit_index_reg* começa em 15 e vai diminuindo até 0. O sinal *cycle_count* controla os 5 ciclos de clock que compõem um único intervalo de bit.

```

signal bit_index_reg : integer range 0 to N_BITS - 1 := N_BITS - 1;
signal cycle_count   : integer range 0 to CYCLES_PER_BIT - 1 := 0;
signal active        : std_logic := '0';
signal out_reg       : std_logic := '0';

nrz_out <= out_reg;
bit_indx <= bit_index_reg;

```

No momento em que o sinal *start* é capturado, a transmissão é iniciada pelo bit mais significativo, *data_in*(15). Em seguida, a cada 5 ciclos de clock, o índice é decrementado e a saída recebe o próximo bit da sequência.

```

if start = '1' and active = '0' then
  active      <= '1';
  bit_index_reg <= N_BITS - 1;
  cycle_count <= 0;
  out_reg     <= data_in(N_BITS - 1);

elsif active = '1' then
  if cycle_count = CYCLES_PER_BIT - 1 then
    cycle_count <= 0;

    if bit_index_reg = 0 then
      active <= '0';
    else
      bit_index_reg <= bit_index_reg - 1;
      out_reg      <= data_in(bit_index_reg - 1);
    end if;
  else
    cycle_count <= cycle_count + 1;
  end if;
end if;

```

A linha *out_reg <= data_in(bit_index_reg - 1)* é a principal característica da modulação NRZ-L implementada: a saída copia diretamente o valor do próximo bit. Portanto, não há cálculo de transição nem comparação com o nível anterior; o nível da onda é simplesmente o valor lógico do bit.

8.3 Trechos relevantes do testbench tb_nrz_l.vhd

No testbench, o tempo é definido por constantes. O clock foi configurado com 20 ns e o intervalo de bit com 100 ns. A constante *CYCLES_PER_BIT* é calculada a partir desses dois valores e enviada ao source por meio do generic map.

```
constant N_BITS : integer := 16;

constant CLK_PERIOD : time := 20 ns;
constant BIT_PERIOD : time := 100 ns;

constant CYCLES_PER_BIT : integer := BIT_PERIOD / CLK_PERIOD;

constant INPUT_BITS : std_logic_vector(N_BITS - 1 downto 0) :=
"1011001110001111";
```

O processo de clock alterna o sinal *clk* a cada metade do período. Como *CLK_PERIOD* vale 20 ns, o clock fica 10 ns em zero e 10 ns em um, formando um ciclo completo a cada 20 ns.

```
clk_process: process
begin
    while true loop
        clk <= '0';
        wait for CLK_PERIOD / 2;
        clk <= '1';
        wait for CLK_PERIOD / 2;
    end loop;
end process;
```

O processo de estímulo aplica reset, libera o reset, aciona start por um ciclo de clock e aguarda o tempo necessário para que os 16 bits sejam transmitidos. Como cada bit dura 100 ns, a transmissão dos dados consome 1600 ns.

```
rst <= '1';
start <= '0';
wait for 40 ns;

rst <= '0';
wait for 40 ns;

wait until rising_edge(clk);
start <= '1';

wait until rising_edge(clk);
start <= '0';

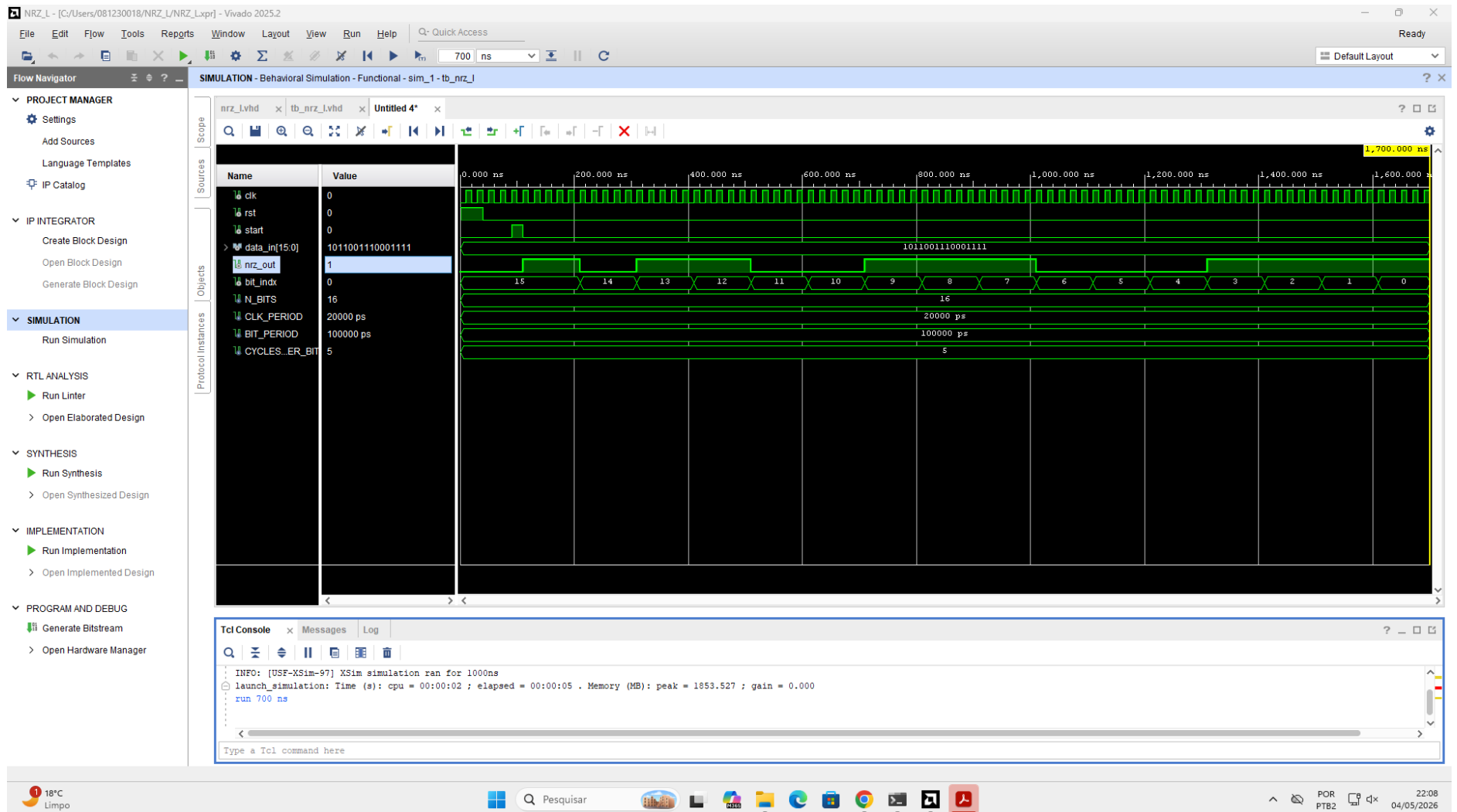
wait for N_BITS * BIT_PERIOD;
```

8.4 Resultado observado no NRZ-L

Na forma de onda do projeto NRZ-L, observa-se que *data_in* permanece constante com a sequência de 16 bits. O sinal *bit_indx* percorre os valores de 15 até 0, e o sinal *nrz_out*

acompanha diretamente o valor do bit associado a cada índice. Quando o índice aponta para um bit 1, a saída permanece alta; quando aponta para um bit 0, a saída permanece baixa.

Figura 1 - Forma de onda da simulação do projeto NRZ-L no Vivado.



Fonte: Vivado Simulator, com base no projeto NRZ-L desenvolvido em VHDL.

8.5 Identificação dos sinais no NRZ-L

A entrada de bits é o sinal `data_in[15:0]`. Na captura, ele aparece como um barramento contendo a sequência completa. A saída modulada é o sinal `nrz_out`. Para interpretar corretamente a onda, deve-se observar simultaneamente `data_in`, `bit_indx` e `nrz_out`.

ELEMENTO OBSERVADO	SINAL NO WAVEFORM	INTERPRETAÇÃO
Entrada de bits	<code>data_in[15:0]</code>	Barramento de 16 bits que armazena a sequência a ser modulada.
Índice do bit	<code>bit_indx</code>	Mostra qual posição do vetor está sendo transmitida em cada intervalo de 100 ns.
Saída modulada	<code>nrz_out</code>	No NRZ-L, copia diretamente o valor do bit apontado por <code>bit_indx</code> .

9 PROJETO NRZ-I

9.1 Princípio de funcionamento

Na modulação Unipolar NRZ-I, o bit não é representado diretamente pelo nível absoluto da onda, mas pela ocorrência ou não de uma transição. Quando o bit de entrada é 1, ocorre inversão do nível do sinal no início do intervalo de bit. Quando o bit de entrada é 0, o nível anterior é mantido.

Na implementação analisada, o primeiro nível de saída é iniciado com o primeiro bit transmitido, `data_in(15)`. A partir dos bits posteriores, a regra passa a ser aplicada: bit 1 inverte o nível anterior e bit 0 mantém o nível anterior. Essa escolha deixa explícito no waveform o ponto inicial da modulação e facilita a comparação com o índice `bit_indx`.

9.2 Trechos relevantes do source `nrz_i.vhd`

A entidade do NRZ-I possui estrutura semelhante à do NRZ-L. Os sinais de entrada e saída são os mesmos, o que facilita a comparação entre as duas modulações. A principal diferença está na forma como `out_reg` é atualizado.

```
entity nrz_i is
  generic (
    N_BITS      : integer := 16;
    CYCLES_PER_BIT : integer := 5
  );
  port (
    clk      : in  std_logic;
    rst      : in  std_logic;
    start    : in  std_logic;
    data_in  : in  std_logic_vector(N_BITS - 1 downto 0);
    nrz_out  : out std_logic;
    bit_indx : out integer range 0 to N_BITS - 1
  );
end nrz_i;
```

No início da transmissão, o registrador de saída recebe o bit mais significativo. Essa decisão define o nível inicial da onda e, a partir desse ponto, os bits seguintes passam a ser interpretados como comandos de inversão ou manutenção do nível.

```
if start = '1' and active = '0' then
    active      <= '1';
    bit_index_reg <= N_BITS - 1;
    cycle_count  <= 0;

    -- Primeiro nível da onda de resposta
    out_reg <= data_in(N_BITS - 1);
```

A parte central do NRZ-I está no comando condicional abaixo. Quando o próximo bit é 1, o valor de out_reg é invertido por meio do operador not. Quando o próximo bit é 0, o próprio valor anterior é mantido. Portanto, a informação passa a estar na transição, e não apenas no nível alto ou baixo.

```
if data_in(bit_index_reg - 1) = '1' then
    out_reg <= not out_reg;
else
    out_reg <= out_reg;
end if;
```

Esse comportamento faz com que duas entradas iguais possam produzir saídas diferentes, dependendo do histórico do sinal. Por exemplo, um bit 1 não significa necessariamente saída em nível alto; significa inversão do nível anterior. Da mesma forma, um bit 0 não significa saída em nível baixo; significa manter o nível já existente.

9.3 Trechos relevantes do testbench tb_nrz_i.vhd

O testbench do NRZ-I segue a mesma lógica temporal do NRZ-L: clock de 20 ns, intervalo de bit de 100 ns e, conseqüentemente, 5 ciclos de clock por bit. Essa igualdade de configuração permite comparar as duas formas de onda de maneira justa.

```
constant CLK_PERIOD : time := 20 ns;
constant BIT_PERIOD : time := 100 ns;

constant CYCLES_PER_BIT : integer := BIT_PERIOD / CLK_PERIOD;

constant INPUT_BITS : std_logic_vector(N_BITS - 1 downto 0) :=
    "1011001110001111";
```

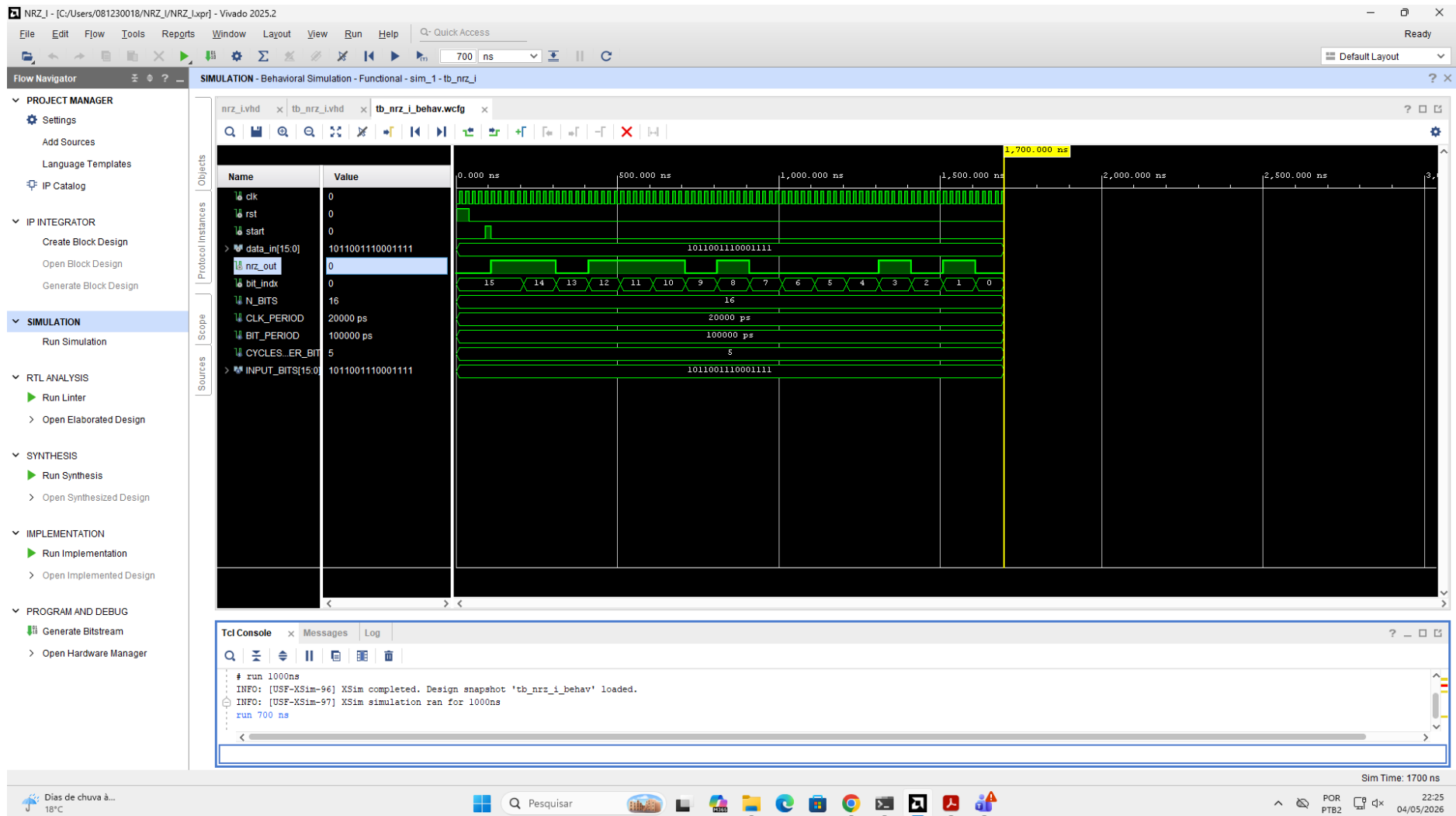
A instância do componente em teste é feita por meio de entity work.nrz_i. O generic map envia para o source a quantidade de bits e a quantidade de ciclos por bit. O port map conecta os sinais gerados no testbench às portas do circuito principal.

```
 uut: entity work.nrz_i
   generic map (
     N_BITS      => N_BITS,
     CYCLES_PER_BIT => CYCLES_PER_BIT
   )
   port map (
     clk      => clk,
     rst      => rst,
     start    => start,
     data_in  => data_in,
     nrz_out  => nrz_out,
     bit_indx => bit_indx
   );
```

9.4 Resultado observado no NRZ-I

Na forma de onda do projeto NRZ-I, o sinal *nrz_out* não copia diretamente os bits de *data_in*. Em vez disso, a saída muda de nível apenas quando o bit associado ao intervalo é 1. Quando o bit associado é 0, a saída permanece no mesmo nível anterior. Esse comportamento comprova que a modulação está baseada na transição do sinal.

Figura 2 - Forma de onda da simulação do projeto NRZ-I no Vivado.



Fonte: Vivado Simulator, com base no projeto NRZ-I desenvolvido em VHDL.

9.5 Identificação dos sinais no NRZ-I

A entrada de bits continua sendo `data_in[15:0]`, e o índice de referência continua sendo `bit_indx`. A diferença está na interpretação da saída modulada. No NRZ-I, `nrz_out` deve ser lido considerando o nível anterior. Assim, a análise exige observar a sequência temporal e não apenas o valor isolado do bit.

ELEMENTO OBSERVADO	SINAL NO WAVEFORM	INTERPRETAÇÃO
Entrada de bits	<code>data_in[15:0]</code>	Barramento de 16 bits que contém a sequência aplicada ao modulador.
Índice do bit	<code>bit_indx</code>	Indica qual bit está sendo analisado em cada intervalo de 100 ns.
Saída modulada	<code>nrz_out</code>	No NRZ-I, inverte quando o bit é 1 e mantém quando o bit é 0.

10. TABELA DE INTERPRETAÇÃO DOS RESULTADOS

A tabela a seguir resume a interpretação da sequência observada nas formas de onda. Para o NRZ-L, a saída é igual ao próprio bit. Para o NRZ-I, o primeiro nível foi iniciado com `data_in(15)`, e os bits seguintes indicam inversão ou manutenção do nível anterior.

bit_indx	Bit de entrada	Saída NRZ-L	Saída NRZ-I	Interpretação
15	1	1	1	Nível inicial definido pelo primeiro bit.
14	0	0	1	Mantém no NRZ-I.
13	1	1	0	Inverte no NRZ-I.
12	1	1	1	Inverte no NRZ-I.
11	0	0	1	Mantém no NRZ-I.
10	0	0	1	Mantém no NRZ-I.
9	1	1	0	Inverte no NRZ-I.
8	1	1	1	Inverte no NRZ-I.
7	1	1	0	Inverte no NRZ-I.
6	0	0	0	Mantém no NRZ-I.
5	0	0	0	Mantém no NRZ-I.
4	0	0	0	Mantém no NRZ-I.
3	1	1	1	Inverte no NRZ-I.
2	1	1	0	Inverte no NRZ-I.
1	1	1	1	Inverte no NRZ-I.
0	1	1	0	Inverte no NRZ-I.

11. COMPARAÇÃO ENTRE NRZ-L E NRZ-I

As duas técnicas pertencem à família NRZ, pois não exigem retorno obrigatório ao nível zero dentro do intervalo de símbolo. A diferença fundamental está no que o bit representa. No NRZ-L, o bit representa diretamente o nível da onda. No NRZ-I, o bit representa uma ação sobre o nível anterior: inverter ou manter.

CRITÉRIO	NRZ-L	NRZ-I
Representação do bit 1	Nível alto durante todo o intervalo de	Inversão do nível anterior no início do

CRITÉRIO	NRZ-L	NRZ-I
	bit.	intervalo.
Representação do bit 0	Nível baixo durante todo o intervalo de bit.	Manutenção do nível anterior.
Dependência do histórico	Não depende do nível anterior; basta ler o bit atual.	Depende do nível anterior para determinar a saída atual.
Interpretação da saída	Direta: saída igual ao bit.	Temporal: saída depende de transições ao longo do tempo.
Uso do bit_idx	Associa o nível alto/baixo ao bit atual.	Associa a ocorrência de inversão ou manutenção ao bit atual.
Facilidade de leitura visual	Mais simples, pois o nível corresponde ao bit.	Exige acompanhar a sequência, pois o mesmo nível pode representar diferentes históricos.

No contexto da simulação desenvolvida, o NRZ-L é mais intuitivo para visualizar, pois a forma de onda de saída reproduz a sequência binária. O NRZ-I, por outro lado, evidencia a ideia de codificação por transição. Essa diferença é observada claramente quando um bit 1 aparece: no NRZ-L a saída apenas assume nível alto, enquanto no NRZ-I a saída alterna entre alto e baixo em relação ao estado anterior.

12. INTERPRETAÇÃO FINAL DOS RESULTADOS

Os resultados obtidos confirmam o funcionamento dos dois moduladores. No NRZ-L, a saída *nrz_out* acompanha diretamente o valor lógico do bit apontado por *bit_idx*. Isso significa que cada trecho da onda pode ser interpretado sem consultar os trechos anteriores: nível alto representa 1 e nível baixo representa 0.

No NRZ-I, a saída *nrz_out* deve ser interpretada de forma sequencial. A partir do nível inicial, cada bit 1 provoca uma inversão, e cada bit 0 preserva o nível anterior. Portanto, a forma de onda final representa a sequência de entrada por meio de transições, não por meio de níveis absolutos.

A parametrização adotada também está correta. O testbench define o tempo físico por meio de *CLK_PERIOD* e *BIT_PERIOD*, enquanto o circuito principal recebe apenas *CYCLES_PER_BIT*. Como *BIT_PERIOD* vale 100 ns e *CLK_PERIOD* vale 20 ns, cada bit permanece estável por exatamente 5 ciclos de clock. Essa separação entre tempo de simulação e ciclos síncronos torna o projeto mais organizado e coerente com a natureza digital do circuito.

Por fim, os sinais *rst*, *start*, *data_in*, *nrz_out* e *bit_idx* são suficientes para compreender o fluxo da simulação. O reset coloca o sistema em estado inicial, o start inicia a transmissão, *data_in* armazena a sequência, *bit_idx* identifica o bit em análise e *nrz_out* apresenta o resultado modulado. Assim, a simulação comportamental no Vivado permite validar completamente os dois projetos antes de qualquer implementação física na FPGA.